

Tecnológico de Monterrey

Evidencia 2 - Actividad Integral de Estructuras de Datos Lineales

Programación de estructuras de datos y algoritmos fundamentales (Gpo 607)

Gabriela Michelle Lagos Aguilar A01648192

Fecha de entrega:

18 oct 2026

Evidencia de ejecución 2
Capturas de pantalla del programa corriendo cada opción del menú, y de una parte del archivo output607.out. 2
Salida de tu programa de pruebas de la Parte A (estructura vacía, un solo elemento, extremos, orden LIFO y FIFO, recorrido hacia adelante y hacia atrás) 6
Casos de borde: archivo inexistente, línea con formato inválido, día sin registros, formato de día inválido, buscar antes de ordenar, ordenar una lista ya ordenada, y empates en las estadísticas. 6
Resultados del ordenamiento 8
Una tabla con tu corrida sobre log607-2.txt y sobre log607-1.txt: predicción, tiempo, comparaciones, profundidad máxima de recursión y si tu implementación conservó el orden de llegada de los registros con la misma fecha y hora 8
Una interpretación breve: qué observaste en cada archivo y por qué 9
Prompts utilizados 9
Reflexión personal 10

Evidencia de ejecución

Capturas de pantalla del programa corriendo cada opción del menú, y de una parte del archivo output607.out.

Para las capturas, tomé el archivo log607-1.txt

```
Archivo a cargar (log607-1.txt o log607-2.txt): log607-1.txt
Cargados: 6818 | Omitidos: 0
```

Orden por fecha:

```
MENU
1. Ordenar por fecha
2. Buscar por dia
3. Mostrar dia descendente
4. Estadisticas
5. Salir
> 1
Prediccion de comparaciones: 60

RESULTADOS MERGE SORT
Tiempo: 89.72 ms
Comparaciones: 78330
Profundidad recursion: 6817
Prediccion: 60
Archivo output608.out generado.
```

Fragmento del archivo resultante más abajo

Buscar por día

```
MENU
1. Ordenar por fecha
2. Buscar por dia
3. Mostrar dia descendente
4. Estadisticas
5. Salir
> 2
Mes (primer tres letras: Sep, Oct, etc.): Sep
Dia: 27
Year: 2024
Sep 27 2024 01:25:59 10.14.16.25 Unencrypted communication
Sep 27 2024 03:25:27 10.14.7.36 Suspicious IP Addresses
Sep 27 2024 08:07:53 10.14.183.50 Load balancing issues
Sep 27 2024 10:02:29 10.14.144.133 Malware Detection Alerts
Sep 27 2024 15:11:08 10.14.218.166 Unauthorized Access Attempts
Sep 27 2024 23:18:02 10.14.170.221 Weak or easily guessable passwords
Total encontrados: 6
```

Mostrar día descendente:

```
MENU
1. Ordenar por fecha
2. Buscar por dia
3. Mostrar dia descendente
4. Estadisticas
5. Salir
> 3
Mes (primer tres letras: Sep, Oct, etc.): Oct
Dia: 21
Year: 2025
Oct 21 2025 23:02:42 10.14.63.125 Social engineering attacks
Oct 21 2025 21:47:03 10.14.101.251 Suspicious IP Addresses
Oct 21 2025 21:18:23 10.14.234.7 Unauthorized Access Attempts
Oct 21 2025 19:29:35 10.14.252.187 Weak or easily guessable passwords
Oct 21 2025 14:31:42 10.14.2.33 Weak or easily guessable passwords
Oct 21 2025 12:41:34 10.14.176.211 Port Scanning Activity
Oct 21 2025 07:56:35 10.14.20.109 Broken authentication and session management
Oct 21 2025 06:45:17 10.14.53.166 Suspicious IP Addresses
Oct 21 2025 03:33:54 10.14.211.47 Suspicious IP Addresses
Oct 21 2025 02:50:53 10.14.103.57 SQL injection
Total encontrados: 10
```

Estadísticas

```
ESTADISTICAS
Total validos: 6818
Dias con mas incidentes (21):
Jan 27 2025
Eventos mas frecuentes (342):
Malware Detection Alerts
IPs mas frecuentes (3):
10.14.123.164
10.14.126.198
10.14.161.42
10.14.162.205
10.14.176.221
10.14.186.139
10.14.199.224
10.14.200.76
10.14.242.186
10.14.29.229
10.14.36.7
10.14.69.71
10.14.94.165
```

Salida

```
MENU
1. Ordenar por fecha
2. Buscar por dia
3. Mostrar dia descendente
4. Estadisticas
5. Salir
> 5
Bye bye
```

Parte de archivo output607.out:



Salida de tu programa de pruebas de la Parte A (estructura vacía, un solo elemento, extremos, orden LIFO y FIFO, recorrido hacia adelante y hacia atrás).

```
PARTE A

1. LINKED LIST
Remover en lista vacia: Excepcion: La lista esta vacia
Un elemento (addFirst 10): 10
Varios elementos (addLast 20, addFirst 5): 5 10 20
Tamano: 3
Remover primero (5): 10 20

2. STACK
Pop en pila vacia: Excepcion: El Stack esta vacio. No se puede hacer pop.
Pila vacia? Si
Push(100) -> Peek: 100, Size: 1
Push(200) -> Pop: 200
Peek actual: 100

3. QUEUE
Dequeue en fila vacia: Excepcion: El Queue esta vacio. No se puede hacer dequeue.
Fila vacia? Si
Enqueue(1) -> Front: 1, Size: 1
Enqueue(2) -> Dequeue: 1
Front actual: 2

4. DOUBLE LINKED LIST
Lista vacia forward:
AddFirst(50): 50
AddLast(150): 50 150
Insertar 100 despues del head: 50 100 150
Backward (tail a head): 150 100 50
Eliminar nodo 100: 50 150
Tamano final: 2

Bye bye.
```

Casos de borde: archivo inexistente, línea con formato inválido, día sin registros, formato de día inválido, buscar antes de ordenar, ordenar una lista ya ordenada, y empates en las estadísticas.

Archivo inexistente:

```
Archivo a cargar (log607-1.txt o log607-2.txt): lalalala
No se encontro el archivo. Intenta de nuevo.

Archivo a cargar (log607-1.txt o log607-2.txt):
```

Línea con formato invalido:

```
MENU
1. Ordenar por fecha
2. Buscar por dia
3. Mostrar dia descendente
4. Estadisticas
5. Salir
> h
Entrada no valida.
```

Formato de día inválido

```
Mes (primer tres letras: Sep, Oct, etc.): sep
Dia: 27
Year: 2024
Total encontrados: 0
```

Día sin registro:

```
Mes (primer tres letras: Sep, Oct, etc.): Feb
Dia: 27
Year: 2006
Total encontrados: 0
```

Buscar antes de ordenar:

```
MENU
1. Ordenar por fecha
2. Buscar por dia
3. Mostrar dia descendente
4. Estadisticas
5. Salir
> 2
Ordena los datos primero.

MENU
1. Ordenar por fecha
2. Buscar por dia
```

Ordenar una lista ya ordenada:

```
MENU
1. Ordenar por fecha
2. Buscar por dia
3. Mostrar dia descendente
4. Estadisticas
5. Salir
> 1
Ya esta ordenado.
```

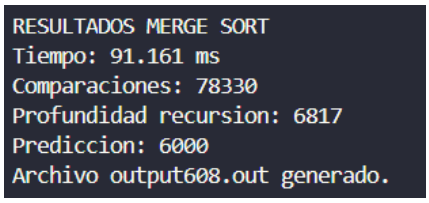
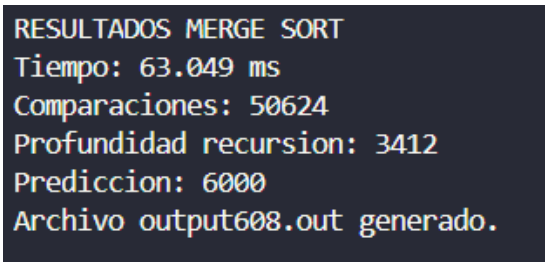
Empates en estadística:

```
ESTADISTICAS
Total validos: 6818
Dias con mas incidentes (21):
Jan 27 2025
Eventos mas frecuentes (342):
Malware Detection Alerts
IPs mas frecuentes (3):
10.14.123.164
10.14.126.198
10.14.161.42
10.14.162.205
10.14.176.221
10.14.186.139
10.14.199.224
10.14.200.76
10.14.242.186
10.14.29.229
10.14.36.7
10.14.69.71
10.14.94.165
```

Resultados del ordenamiento

Una tabla con tu corrida sobre log607-2.txt y sobre log607-1.txt: predicción, tiempo, comparaciones, profundidad máxima de recursión y si tu implementación conservó el orden de llegada de los registros con la misma fecha y hora.

Función	log607-1.txt	log605-2.txt
predicción	O(n log n)	O(n log n)

Tiempo	91.161 ms	63.049 ms
Comparaciones	78330	50624
Profundidad máxima	6817	3412
¿Conservó orden de llegada?	SI	Si
Consola		

Una interpretación breve: qué observaste en cada archivo y por qué.

Los resultados de ambos archivos al final no son extremadamente diferentes. En cuestión de tiempo, la diferencia es muy poca, y se debe a cómo funciona merge sort (no importa qué tan ordenado esté, va a hacer toda su organización).La variación en el número de comparaciones y la profundidad máxima no se debe a un cambio de eficiencia en el algoritmo, sino a que los dos archivos tienen una cantidad total diferente de líneas de texto a procesar.

Prompts utilizados

Me pasa mucho que cuando hago copy paste de tareas pasadas en un nuevo archivo, hago muchos errores por la manera que pego las cosas, como fallas de lógica y muchos problemas de sintaxis.

1. “What is the problem?” y le mando el texto de la consola y la línea donde sale el error.
2. “How can I structure LogRecord so that the dates are compared chronologically?” me generó una estructura de la clase LogRecord y cómo se tenían que ver las sobrecargas de funciones. Tuve que ajustar el parseo de la cadena de texto con stringstream para manejar la concatenación, también regresando a la evidencia pasada para recordar cómo lo había hecho.
3. “Is this correct” y le enviaba mi código. Quizás fue que no le expliqué muy bien la idea, pero no detectó un error de puntero que había hecho. En la opción de mostrar los días en orden

descendente, me salía un error ya que no conecté los nodos entonces quedaban sueltos. Fue hasta que le pregunté cuál era el problema después de correr el código que lo logró identificar.

4. “Could you compile instructions of usage in the README” para que me diera las especificaciones de correr el código.

Reflexión personal

Tiendo a mandarle mi código a la IA cuando o no sé qué hacer, siento que le falta algo, o ya pienso que la idea es suficientemente sólida como para correr. Busco que me corrija mis posibles errores y que me explique bien cómo resolverlos para no volverlos a hacer. Ocupé mucho de mi código viejo para ambas partes, y me dediqué a que estuviera bien unido antes de mandar. Hice algunos errores de sintaxis en la migración de código, y luego también el error de puntero. Sin asistencia, el desarrollo me hubiera tomado muchos más intentos y fallas y quizás hubiera sido más cuidadosa con mi código si es fuera el caso.

Para guardar los datos principales de los logs, usé la lista doblemente ligada porque me permite agregar registros dinámicamente sin preocuparme por tener un tamaño fijo desde el inicio. Para la opción de mostrar los registros en orden descendente, mi lista de por sí ya se puede recorrer hacia atrás gracias al apuntador prev, lo cual tiene la enorme ventaja de no gastar memoria extra. Sin embargo, decidí implementar una Pila para este paso. Lo hice porque la Pila me permite ir sacando y guardando únicamente los registros que coinciden con la fecha exacta que busco, para luego imprimirlos de forma aislada. La desventaja principal de usar la Pila es que obviamente usa memoria adicional proporcional a lo que encuentre, pero a cambio organiza los resultados de una forma mucho más limpia y segura sin tocar los enlaces de mi lista original.

Sobre la búsqueda binaria, me di cuenta de que no se puede aplicar directamente sobre una lista ligada porque este algoritmo necesita a fuerzas tener acceso aleatorio. Es decir, necesita poder saltar al índice de la mitad al instante, justo como se hace en un arreglo. En una lista ligada no puedes hacer eso, ya que para llegar al nodo de en medio tienes que ir avanzando uno por uno desde el principio, lo que hace que se pierda toda la velocidad que hace tan bueno a este algoritmo. Para que fuera posible implementarlo, el costo de procesamiento sería muy alto, porque el programa tendría que copiar temporalmente todos y cada uno de los datos de mi lista hacia un arreglo estático antes de poder buscar.

Finalmente, viendo las mediciones en los archivos, noté que mi algoritmo se comportó casi idéntico en ambos. Esto pasa porque el algoritmo de Merge Sort siempre divide los datos a la mitad de forma

recursiva, sin importarle en lo absoluto si el archivo de texto ya venía un poco ordenado o completamente revuelto. La profundidad de recursión me indica precisamente cuántas llamadas a la función se quedaron en espera en la pila de llamadas de la computadora antes de empezar a juntar los nodos de regreso. Al hacer exactamente el mismo número de divisiones matemáticas siempre por su naturaleza de divide y vencerás, el algoritmo mantiene un tiempo de ejecución muy estable sin importar la situación.